Chapter 1: Project Vision

1.1 Problem Statement

Urdu, being one of the most widely spoken languages in South Asia with over 170 million native speakers, still lacks high-quality Text-to-Speech (TTS) systems capable of producing natural, human-like speech with personalization capabilities. Current solutions produce robotic or unnatural voices, have limited language and dataset support, and do not allow users to clone or customize voices. These limitations significantly affect accessibility, content creation, and user engagement for Urdu-speaking communities worldwide. This highlights the urgent need for an Urdu TTS system capable of generating natural-sounding speech using state-of-the-art deep learning methods and supporting personalized voice cloning.

1.2 Business Opportunity

Developing a high-quality Urdu TTS system with voice cloning presents a significant business opportunity by filling a major gap in the market. Millions of Urdu speakers worldwide could benefit from personalized, natural-sounding speech solutions for digital learning, content consumption, and accessibility. This system gains competitive advantage by offering features not widely available in existing Urdu TTS solutions, such as zero-shot voice cloning and a fully web-based, user-friendly interface powered by a fine-tuned ChatterBox TTS v0.1.2 model.

Potential applications include:

- Audiobooks and educational content for Urdu-speaking students and learners.
- Accessibility tools for visually impaired Urdu speakers.
- Virtual assistants capable of interacting in Urdu with personalized voices.
- Call centers, where automated agents interact with customers in natural, personalized Urdu speech, improving customer experience and operational efficiency.

1.3 Objectives

- To build a natural-sounding Urdu Text-to-Speech (TTS) system using ChatterBox TTS v0.1.2.
- To fine-tune ChatterBox TTS v0.1.2 on a large-scale 20,000-file Urdu dataset sourced from Kaggle for improved speech quality and Urdu language adaptation.
- To integrate voice cloning using Resemble Perth v1.0.1 for generating speech in a user's own or custom voice.
- To provide an easy-to-use web interface with sidebar dashboard for text input, voice playback, and 3D audio visualization.
- To allow users to play back or download generated audio files in WAV format.
- To ensure the system delivers high intelligibility and naturalness in synthesized Urdu speech.

1.4 Project Scope

1.4.1 In-Scope

1. Urdu Text-to-Speech (TTS) Model: The core functionality focuses exclusively on developing an Urdu TTS system using ChatterBox TTS v0.1.2 fine-tuned on a 20,000-file Urdu dataset from Kaggle, ensuring accurate and high-quality audio output at 24,000 Hz sample rate.
2. Voice Cloning Integration: The system includes integrated voice cloning using Resemble Perth v1.0.1, allowing users to generate personalized Urdu speech by providing a reference audio sample.
3. Web Interface: A user-friendly web-based interface with a sidebar dashboard serves as the primary platform for text input, audio visualization, and TTS feature access.
4. Backend API: A RESTful backend provides user authentication via JWT, TTS inference endpoints, voice cloning endpoints, and generation history tracking.

5. Speech Playback and Download: The web interface allows users to play back and download the generated Urdu speech in WAV format (32-bit float, 24 kHz).
6. Focus on Urdu Language Support: All core features and functionalities specifically target Urdu language support and personalized voice synthesis.

1.4.2 Out-of-Scope

7. Multilingual Support: The system will not include TTS functionality for languages other than Urdu in this phase.
8. Emotional or Expressive Speech Synthesis: Advanced prosody modifications are excluded from this phase and may be addressed in future work.
9. Mobile Application and Offline Version: Development of a native mobile app or offline version is excluded from this phase.
10. Long-term Maintenance and Updates: Ongoing post-deployment updates or support will not be covered in this phase.

1.5 Constraints

- Limited availability of large, high-quality Urdu speech datasets for training deep neural TTS models.
- Potential hardware constraints for training large neural TTS models, addressed by using Google Colab with GPU acceleration.
- Time constraints for fine-tuning and testing the system within two academic semesters.
- The system must remain lightweight enough for deployment on standard computing resources without requiring specialized server hardware.

1.6 Feasibility

1.6.1 Technical Feasibility

The availability of ChatterBox TTS v0.1.2, a state-of-the-art diffusion-based text-to-speech model with T3 Turbo architecture, provides a strong technical foundation for this project. ChatterBox TTS supports high-quality speech synthesis and voice cloning through its integrated Resemble Perth v1.0.1 component, Conformer v0.3.2 acoustic model, and Diffusers v0.29.0 vocoder. The model produces output at 24,000 Hz sample rate in 32-bit float WAV format. Access to a 20,000-file Urdu dataset from Kaggle further supports technical feasibility.

1.6.2 Resource Feasibility

The project leverages Google Colab with GPU acceleration for model fine-tuning, eliminating the need for dedicated hardware. Required open-source libraries include PyTorch 2.6.0, TorchAudio, ChatterBox TTS v0.1.2, Diffusers v0.29.0, Conformer v0.3.2, and Resemble Perth v1.0.1. The development team of three students brings expertise in NLP, deep learning, and full-stack web development. No paid APIs or proprietary datasets are required, ensuring cost-effective implementation.

1.6.3 Time Feasibility

A structured timeline with clear milestones ensures the project is successfully completed within two academic semesters (FYP1 and FYP2). This duration adequately covers all phases: data collection and preprocessing, ChatterBox TTS fine-tuning, full-stack web application development, testing, and final deployment.

1.6.4 Market Feasibility

There is a strong and growing demand for Urdu voice technologies in Pakistan and other Urdu-speaking regions. Educational platforms, accessibility tools, and call centers urgently require natural-sounding Urdu speech for automated systems. Since few competitive solutions exist for high-quality Urdu TTS with voice cloning capabilities, the market potential is high. Sybrid, the industry partner, has expressed interest in evaluating the system for customer service automation, validating commercial viability.

1.7 Stakeholders Description

1.7.1 Stakeholders Summary

Table 1.1 below summarizes the key stakeholders involved in this project.

Table 1.1: Stakeholders Summary

Stakeholder	Role	Responsibilities
Muhammad Bilal, Rohail Ahmad, Fiza Wahab	Development Team	Design, development, training, and testing of the Urdu TTS system.
Dr. Arshad Iqbal	Academic Supervisor	Provides academic guidance and ensures research alignment with institutional standards.
Muhammad Murtaza Khan (Sybrid)	Industry Supervisor	Provides industry insights and ensures the project meets commercial standards.
Urdu-speaking individuals & organizations	End Users	Benefit from natural-sounding Urdu speech synthesis and voice cloning features.
Sybrid	Industry Partner	Potential adopter of the system for call center automation and customer service.

1.7.2 Key High-Level Goals and Problems of Stakeholders

- Development Team: The goal is to deliver a robust, high-performance Urdu TTS system using ChatterBox TTS v0.1.2. Key challenges include managing Urdu data limitations, achieving natural speech quality through fine-tuning, and optimizing the T3 Turbo model for the Urdu language within academic time constraints.
- Supervisors: The aim is to maintain academic and technical excellence, with expectations of timely delivery and comprehensive documentation throughout the project lifecycle.
- End Users: End users seek a simple, accurate, and pleasant Urdu TTS experience. Key concerns include naturalness of the generated voice, ability to personalize or clone voices, and overall usability of the web interface.

- Organization (Sybrid): The organization aims to integrate the Urdu TTS system to improve automation and customer interaction in call centers. Their challenge is ensuring smooth deployment and seamless operation within existing workflows.

Chapter 2: Software Requirements

2.1 Introduction

This chapter presents the software requirements for the Urdu Text-to-Speech (TTS) system. It covers functional and non-functional requirements, feature specifications, use case descriptions with diagrams, and the software development plan. All requirements are defined based on the capabilities of ChatterBox TTS v0.1.2, the Kaggle Urdu 20K dataset, and the full-stack web application architecture.

2.2 Functional Requirements

Table 2.1 lists the functional requirements of the Urdu TTS system.

Table 2.1: Functional Requirements

FR#	Feature	Description
FR-01	**User Registration**	Users create an account with username, email, and password. Passwords are hashed using bcrypt.
FR-02	**User Login & JWT Auth**	Users log in with valid credentials and receive a JWT access token for authenticated API access.
FR-03	**Simple TTS Generation**	Authenticated users input Urdu text and generate speech using the fine-tuned ChatterBox TTS v0.1.2 model.
FR-04	**Voice Cloning TTS**	Users upload a reference audio sample; Resemble Perth v1.0.1 extracts voice embeddings to condition the TTS model.
FR-05	**Audio Playback**	The web interface plays back generated WAV audio with 3D audio visualization during playback.
FR-06	**Audio Download**	Users can download generated speech as WAV files (24 kHz, 32-bit float) for offline use.
FR-07	**Generation History**	The system stores and displays a user's past TTS generations, including text input, audio file, and timestamp.

FR-08	**Account Management**	Users can view and update their profile information and manage account settings.
FR-09	**Theme Toggle**	The web interface supports switching between dark and light modes for improved user experience.
FR-10	**API Health Check**	A health endpoint confirms that the backend and TTS model are loaded and operational.

2.3 Non-Functional Requirements

Table 2.2 presents the non-functional requirements of the system.

Table 2.2: Non-Functional Requirements

NFR#	Category	Requirement
NFR-01	**Performance**	TTS generation should complete within 10 seconds for standard Urdu text inputs on GPU-enabled hardware.
NFR-02	**Audio Quality**	Generated speech should achieve a Mean Opinion Score (MOS) of at least 3.5/5.0 for intelligibility and naturalness.
NFR-03	**Security**	All user data must be protected using JWT authentication. Passwords are hashed with bcrypt. API endpoints require valid tokens.
NFR-04	**Usability**	The web interface must be intuitive for non-technical Urdu-speaking users, with responsive design for desktop and mobile browsers.
NFR-05	**Scalability**	The backend must handle concurrent API requests using asynchronous endpoints with an ASGI server.
NFR-06	**Reliability**	The system must handle invalid input gracefully, returning proper HTTP error codes and meaningful error messages.
NFR-07	**Maintainability**	Code must follow modular architecture separating frontend, backend, and ML inference components.
NFR-08	**Portability**	The system should be deployable on any CUDA-enabled server or cloud environment.

2.4 Features List

Table 2.3 summarizes the main features of the Urdu TTS system.

Table 2.3: Features List

F#	Feature	Description	Priority
F-01	**Urdu TTS Synthesis**	Generate Urdu speech from input text using ChatterBox TTS v0.1.2	High
F-02	**Voice Cloning**	Clone any voice using a short audio sample via Resemble Perth v1.0.1	High
F-03	**3D Audio Visualizer**	Real-time 3D waveform visualization during audio playback	Medium
F-04	**User Authentication**	JWT-based registration and login system with secure password hashing	High
F-05	**Generation History**	Database-backed history of all past TTS generations per user	High
F-06	**Audio Download**	Download generated WAV audio files for offline use	Medium
F-07	**Dark/Light Theme**	Toggle between dark and light UI modes	Low
F-08	**Responsive Design**	Responsive layout for desktop and mobile browsers	Medium

2.5 Use Case Descriptions

2.5.1 Actors

- User (Registered): An authenticated Urdu-speaking individual who accesses TTS generation and voice cloning features via the web dashboard.
- Guest (Unauthenticated): A visitor who can access the home, about, and features pages but must register or log in to use TTS features.
- System: The backend TTS pipeline including ChatterBox TTS v0.1.2, the API server, SQLite database, and Resemble Perth voice encoder.

2.5.2 Use Case List

The system supports eleven use cases organized into three functional groups. Account & UI use cases include: Register/Sign up, Login, Manage account, and Change theme (light/dark). TTS & Voice use cases include: Input Urdu text, Generate speech, Record/upload voice, and Voice cloning. Output use cases include: Playback audio, Download audio (WAV), and View history.

2.5.3 Use Case Diagram

Figure 2.1 illustrates the use case diagram showing all actors and their interactions with the system.

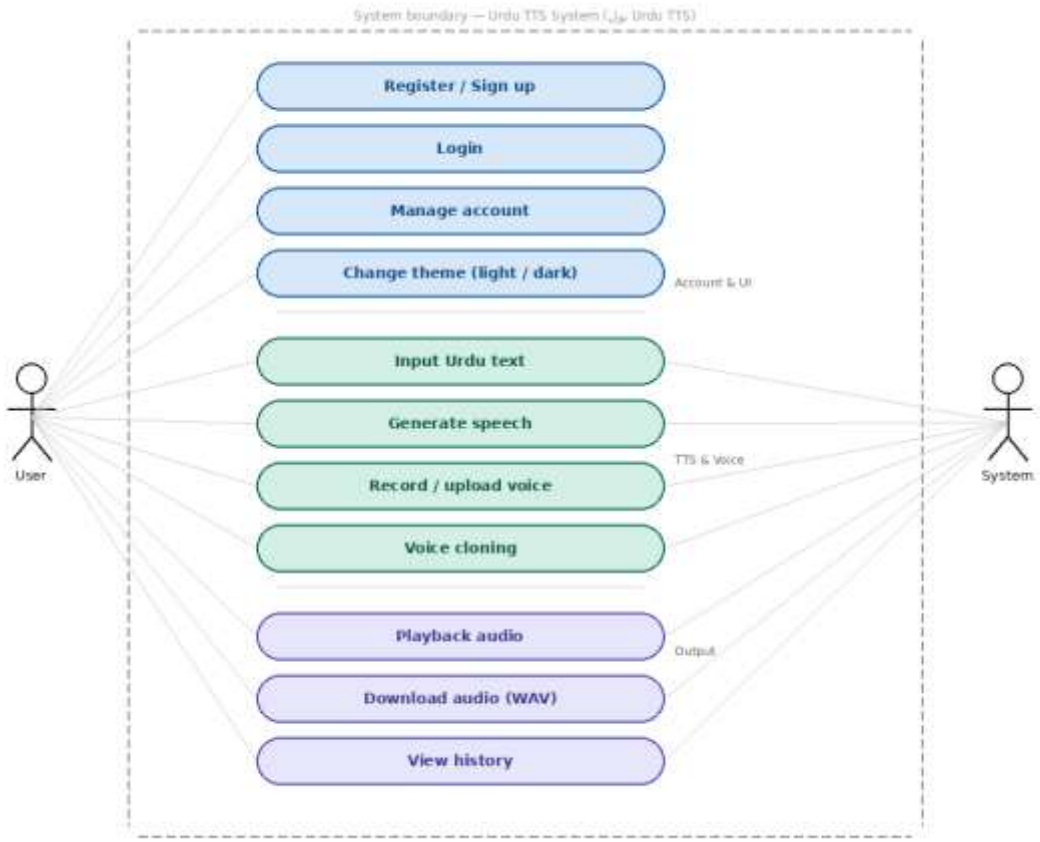

Figure 2.1: Use case diagram — Urdu TTS System (بول Urdu TTS)

2.5.4 Key Use Case Descriptions

Table 2.4 presents the detailed descriptions for the two primary use cases.

Table 2.4: Use Case Descriptions

Field	UC-01: Generate Urdu Speech
Actor	Registered User
Precondition	User is logged in with a valid JWT token.
Main Flow	1. User navigates to TTS Dashboard. 2. User enters Urdu text in the input field. 3. User clicks Generate. 4. Backend validates token and sends text to TTS endpoint. 5. ChatterBox TTS v0.1.2 processes text through S3 Tokenizer, Conformer acoustic model, and Diffusers vocoder. 6. Generated WAV audio is returned to frontend. 7. Audio plays with 3D audio visualizer.
Postcondition	Audio is generated, played, and stored in generation history.
Exception	Invalid or empty Urdu text returns HTTP 400 with an error message.

Field	UC-02: Clone Voice & Generate Speech
Actor	Registered User
Precondition	User is logged in and has a reference audio sample (WAV/MP3).
Main Flow	1. User navigates to Voice Cloning section of the dashboard. 2. User uploads or records a reference audio sample. 3. User enters Urdu text. 4. User clicks Clone & Generate. 5. Backend sends audio and text to clone endpoint. 6. Resemble Perth v1.0.1 extracts voice embeddings. 7. ChatterBox TTS v0.1.2 generates speech conditioned on extracted voice embeddings. 8. Personalized WAV audio is returned to frontend for playback.
Postcondition	Personalized audio is generated, played, and stored in history.
Exception	Invalid audio format or empty text returns HTTP 422 with error details.

2.6 Software Development Plan

2.6.1 Development Methodology

The project follows an Agile development model, ensuring iterative progress, continuous integration, and regular testing of components. Development is organized into sprints covering model fine-tuning, API development, frontend development, and integration testing phases.

2.6.2 Development Phases

Phase 1 — Research & Data Preparation: Literature review of existing Urdu TTS models and selection of ChatterBox TTS v0.1.2 as the core model. Collection and preprocessing of the Kaggle Urdu 20,000-file dataset (filtering to 1.0–10.0 second clips, speaker-level 90/5/5 split).

Phase 2 — Model Fine-Tuning: Setting up the fine-tuning environment on Google Colab with CUDA GPU using the chatterbox-finetuning repository. Training the T3 Turbo model on the preprocessed Urdu dataset and saving fine-tuned weights in safetensors format.

Phase 3 — Backend Development: Building RESTful API endpoints for authentication, TTS inference, voice cloning, and history management. Integrating ChatterBox TTS v0.1.2 inference pipeline and setting up SQLite database for user and history management.

Phase 4 — Frontend Development: Developing a multi-page web application with sidebar dashboard including TextToSpeech, VoiceCloning, History, and Settings components. Integrating 3D audio visualization for real-time waveform display during playback.

Phase 5 — Testing & Deployment: Unit testing of API endpoints, integration testing of TTS pipeline, user acceptance testing with Urdu-speaking participants, and performance evaluation using Word Error Rate (WER), Mean Opinion Score (MOS), and Real-Time Factor (RTF).

2.6.3 Technology Stack

Table 2.5 presents the complete technology stack used in this project.

Table 2.5: Technology Stack

Category	Technology	Version / Details
TTS Model	ChatterBox TTS	v0.1.2 — T3 Turbo diffusion-based architecture
Voice Cloning	Resemble Perth	v1.0.1 — voice embedding extraction
Acoustic Model	Conformer	v0.3.2 — Conformer-based acoustic modeling

Vocoder	Diffusers	v0.29.0 — diffusion-based vocoder
Deep Learning	PyTorch	v2.6.0 — model training and inference
Backend Framework	FastAPI + Uvicorn	Async REST API with ASGI server
Authentication	JWT / OAuth2 Bearer	python-jose, passlib with bcrypt
Database	SQLite + SQLAlchemy	ORM-based data management
Frontend	React 18 + Vite	Component-based SPA with fast HMR
Styling	Tailwind CSS	Utility-first responsive CSS framework
3D Visualization	Three.js	AudioVisualizer3D real-time waveform rendering
Training Platform	Google Colab	GPU-accelerated CUDA training environment
Dataset	Kaggle Urdu Dataset	20,000 Urdu WAV + TSV transcription files

2.6.4 Project Timeline

Table 2.6 presents the project timeline across both FYP semesters.

Table 2.6: Project Timeline

Phase	Duration	Stage	Key Activities
FYP 1	Month 1	Requirements & Research	Gather requirements, review literature, analyze Urdu TTS models, select ChatterBox TTS v0.1.2.
	Month 2	System Design	Design system architecture, UI mockups, database schema, and technical specifications.
	Month 3	Dataset Preparation	Clean, filter, and preprocess 20,000 Urdu audio files; create training manifests.
	Month 4	Model Research & Testing	Research TTS architectures, run initial tests, and confirm ChatterBox TTS v0.1.2 suitability.
FYP 2	Month 5	Model Development	Fine-tune ChatterBox TTS v0.1.2 on Urdu dataset; integrate Resemble Perth voice cloning.
	Month 6	Application Development	Develop and integrate frontend dashboard and backend API with TTS inference pipeline.

	Month 7	Integration & Testing	Integrate all modules, perform system and usability testing, optimize performance.
	Month 8	Deployment & Documentation	Deploy system, finalize documentation, prepare final presentation and demo.

Chapter 3: System Overview

3.1 Introduction

This chapter presents the overall system design and architecture of the Urdu Text-to-Speech (TTS) system (بول Urdu TTS). It describes the high-level system architecture, the ChatterBox TTS v0.1.2 inference pipeline, data flow across system components, entity relationships for the database, and the key design decisions that guide implementation. All diagrams reflect the actual technology stack deployed in this project.

3.2 System Architecture

The system follows a three-tier client-server architecture comprising a frontend layer, a backend layer, and an ML inference layer. The three tiers communicate via RESTful HTTP APIs secured with JWT authentication. Figure 3.1 below provides a high-level overview of the complete system architecture.

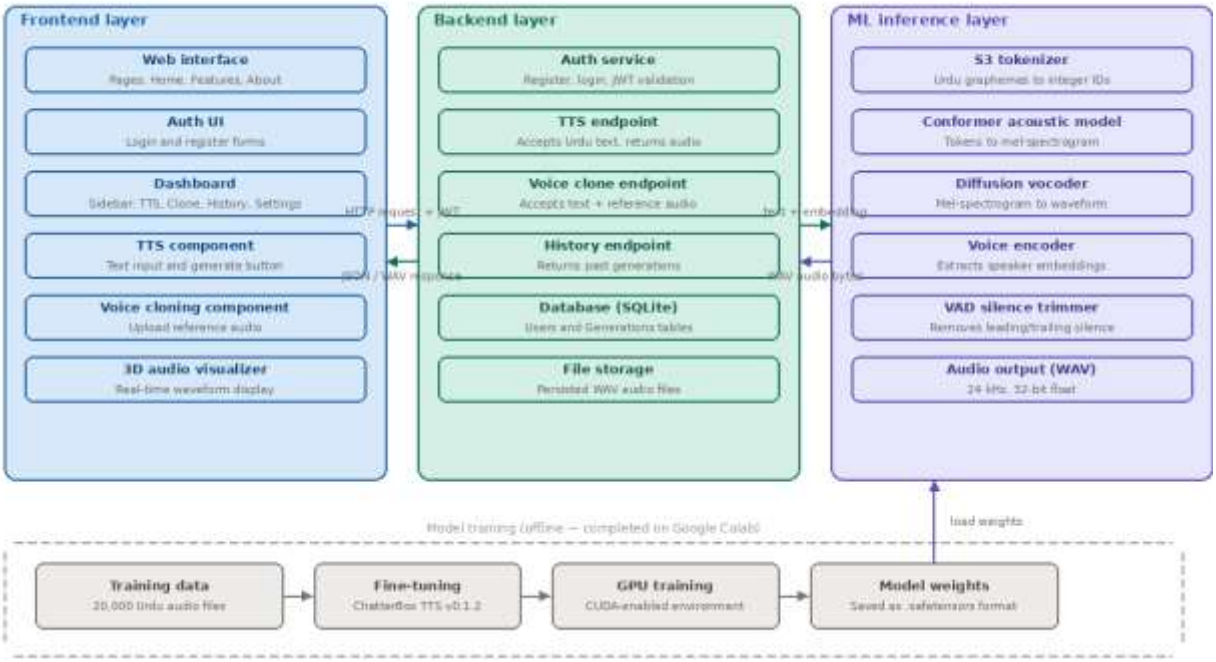

Figure 3.1: System architecture — بول Urdu TTS (three-tier: Frontend, Backend, ML inference)

The frontend layer handles all user interaction: text input, audio playback with 3D visualization, voice cloning file uploads, and generation history display. All user requests are authenticated via JWT Bearer tokens before reaching the backend.

The backend layer processes authenticated API requests. It routes TTS requests to the ML inference pipeline, persists user data and generation history in a SQLite database via an ORM, and returns generated WAV audio to the frontend.

The ML inference layer constitutes the core of the system. It processes Urdu text through the S3 Tokenizer, generates acoustic features via the Conformer v0.3.2 acoustic model, synthesizes audio via the Diffusers v0.29.0 vocoder, and applies voice cloning embeddings extracted by Resemble Perth v1.0.1 when a reference audio is provided. The model weights were produced by fine-tuning ChatterBox TTS v0.1.2 on 20,000 Urdu audio files using Google Colab's GPU environment.

3.3 ChatterBox TTS v0.1.2 Pipeline

Figure 3.2 illustrates the internal pipeline of ChatterBox TTS v0.1.2, showing how Urdu text is transformed into a WAV audio file through a sequence of learned model components.

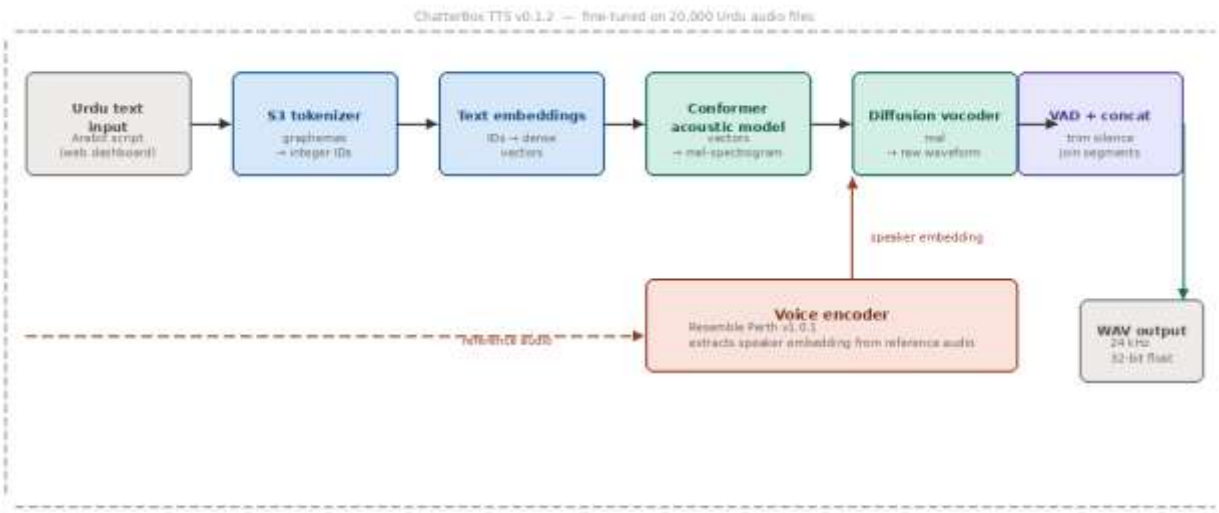

Figure 3.2: ChatterBox TTS v0.1.2 internal pipeline (fine-tuned on 20,000 Urdu audio files)

The pipeline operates in ten steps: (1) User inputs Urdu text via the web interface. (2) The backend validates and preprocesses the text, removing diacritics and splitting it into sentences. (3) Each sentence is tokenized using the S3 Tokenizer with the Urdu grapheme vocabulary. (4) Tokens are converted to dense text embeddings. (5) The Conformer v0.3.2 acoustic model generates mel-spectrogram features. (6) The Diffusers v0.29.0 vocoder converts mel features to raw audio waveforms. (7) If voice cloning is requested, Resemble Perth v1.0.1 extracts voice embeddings from the reference audio and conditions the synthesis. (8) Voice Activity Detection (VAD) trims silence from each segment. (9) Segments are concatenated with 0.2-second pauses. (10) Final audio is saved as a 24 kHz, 32-bit float WAV file and returned via API.

3.4 Data Flow Diagrams

3.4.1 DFD Level 0 — Context Diagram

Figure 3.3 shows the system at Level 0 — the whole system as a single process with external entities. The User provides Urdu text input and an optional voice sample, and receives WAV audio output. The Training dataset represents the 20,000-file Kaggle Urdu corpus used offline during fine-tuning.

Figure 3.3: DFD Level 0 — context diagram

3.4.2 DFD Level 1 — Main Processes

Figure 3.4 decomposes the system into four main processes: P1 (User Management), P2 (TTS Processing), P3 (Voice Cloning), and P4 (History Management), along with two data stores — the Users database and the Generations database.

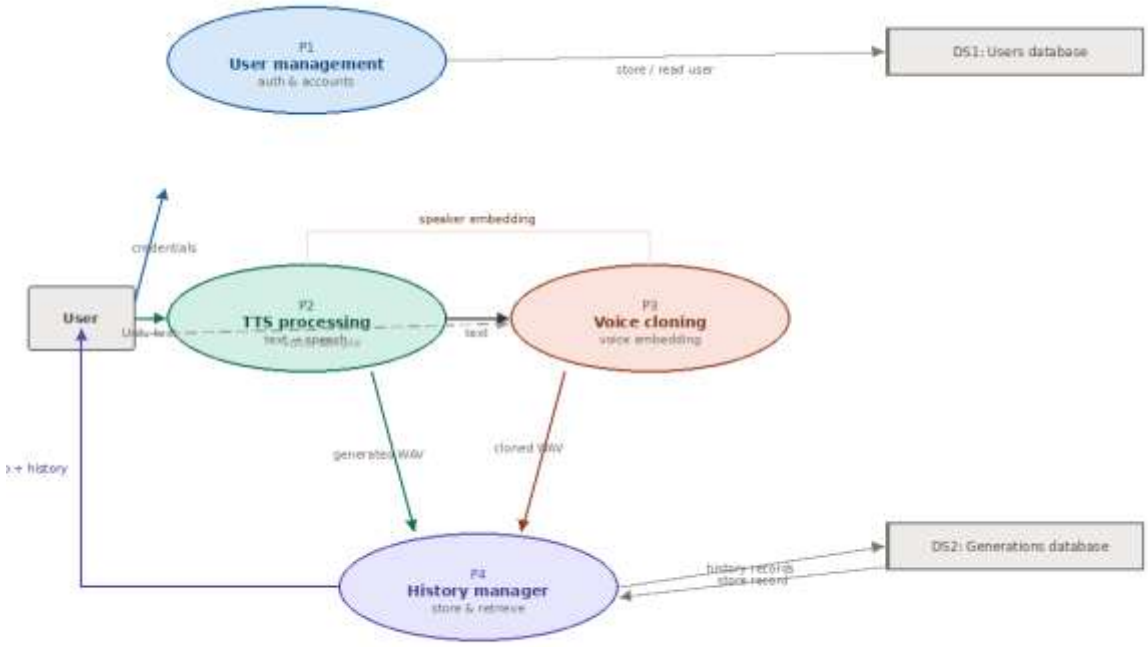

Figure 3.4: DFD Level 1 — main system processes and data stores

3.4.3 DFD Level 2 — TTS Processing Detail

Figure 3.5 zooms into Process P2 (TTS Processing), showing the detailed sub-processes within the ChatterBox TTS v0.1.2 pipeline: text validation, sentence splitting, S3 tokenization, text embedding, Conformer acoustic modeling, optional voice encoder conditioning, Diffusers vocoding, VAD trimming, and audio concatenation.

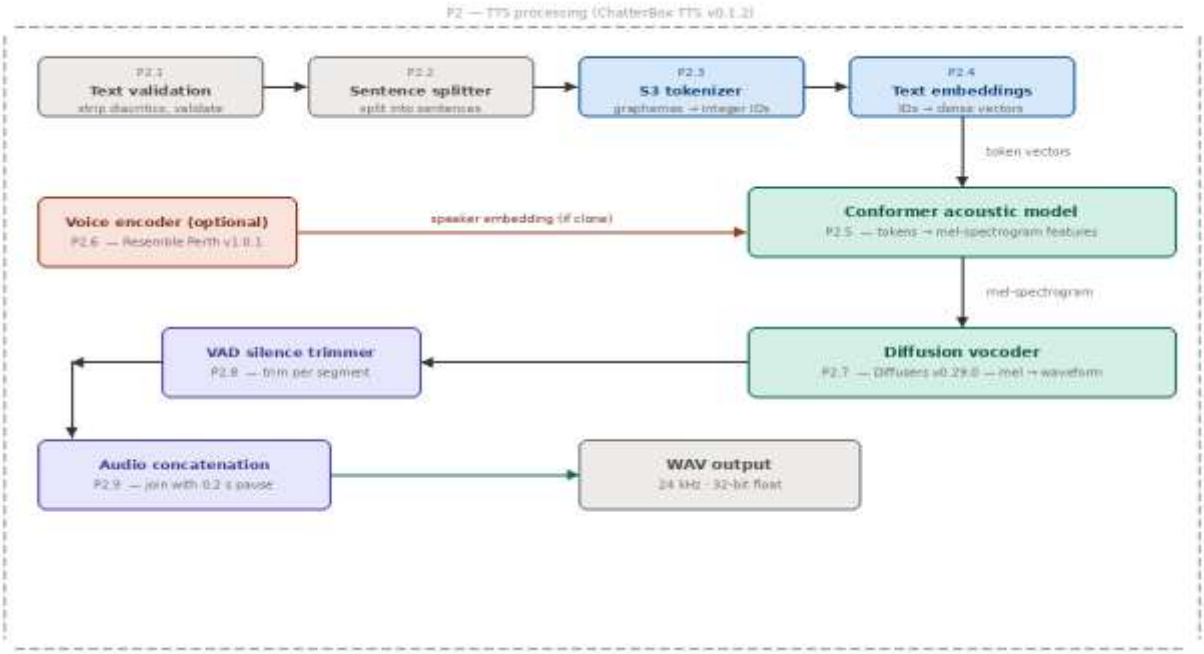

Figure 3.5: DFD Level 2 — P2 TTS processing detail (ChatterBox TTS v0.1.2)

3.5 Sequence Diagram

Figure 3.6 illustrates the runtime message flow between the User, Frontend, Backend, Voice Encoder (Resemble Perth), TTS Model (ChatterBox), and Storage. It shows both the standard TTS flow and the optional voice cloning path.

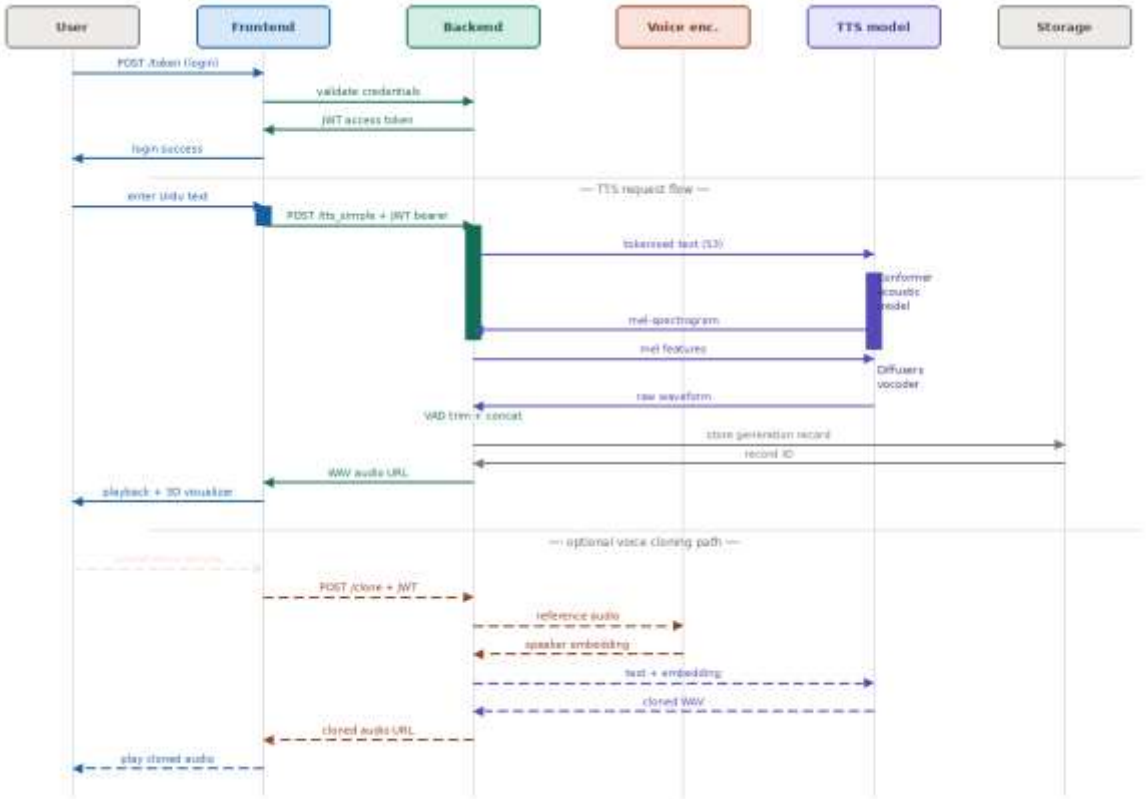

Figure 3.6: Sequence diagram — TTS request flow (standard and voice cloning paths)

3.6 Entity-Relationship Diagram (ERD)

The SQLite database managed by an ORM contains two primary entities: Users and Generations. Table 3.1 and Table 3.2 describe the schema for each entity.

Table 3.1: Users Table Schema

Column	Type	Constraints	Description
id	INTEGER	PK, AutoIncrement	Unique user identifier
username	VARCHAR	NOT NULL, UNIQUE	User's chosen username
email	VARCHAR	NOT NULL, UNIQUE	User's email address
hashed_password	VARCHAR	NOT NULL	bcrypt-hashed password

created_at	DATETIME	NOT NULL	Account creation timestamp
is_active	BOOLEAN	DEFAULT TRUE	Account active status flag

Table 3.2: Generations Table Schema

Column	Type	Constraints	Description
id	INTEGER	PK, AutoIncrement	Unique generation record identifier
user_id	INTEGER	FK → Users.id	Owner of the generation record
input_text	TEXT	NOT NULL	Urdu text input used for generation
audio_path	VARCHAR	NOT NULL	File path to generated WAV output
generation_type	VARCHAR	NOT NULL	Type: 'simple' or 'voice_clone'
created_at	DATETIME	NOT NULL	Timestamp of generation

The relationship between Users and Generations is one-to-many: one user can have many generation records, linked via the user_id foreign key.

3.7 State / Flow Diagram

Figure 3.7 illustrates the step-by-step states that a request passes through, from Urdu text input through the ChatterBox TTS v0.1.2 pipeline to the final WAV audio output. The diagram includes the optional voice cloning decision branch.

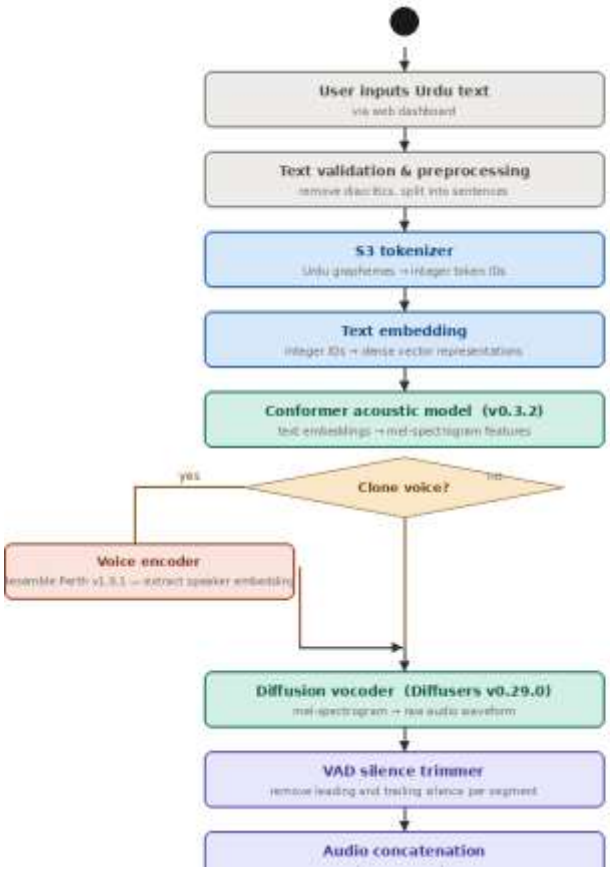

Figure 3.7: State / flow diagram — ChatterBox TTS v0.1.2 processing states

3.8 Component Diagram

Figure 3.8 shows the modular component structure of the system organized into three packages: Frontend, Backend, and ML Inference. Each package contains its constituent components, and dependency arrows show how the packages interact.

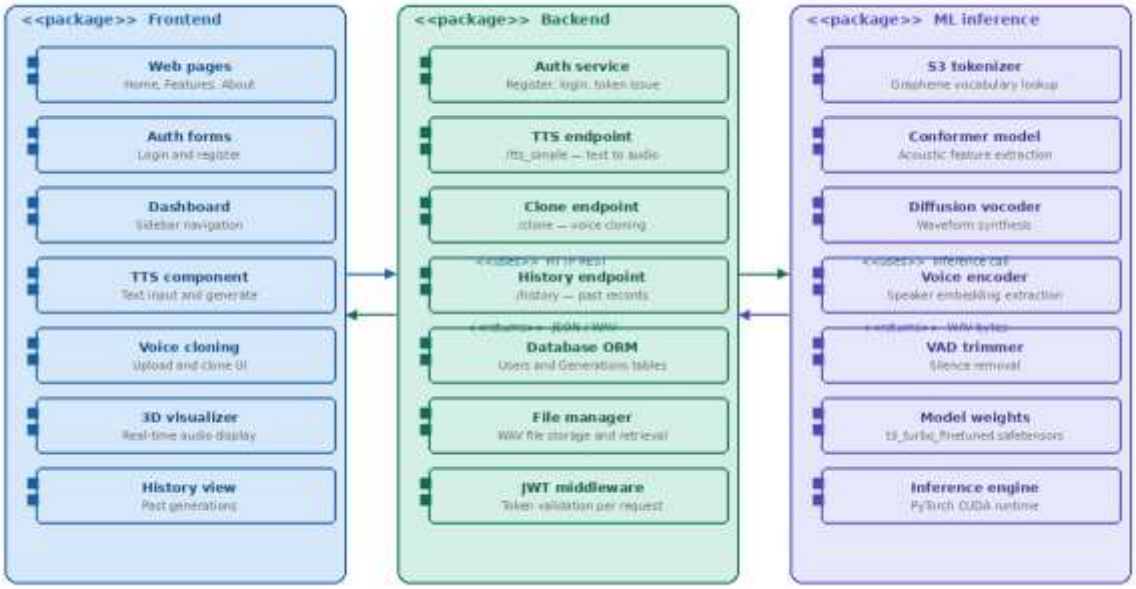

Figure 3.8: Component diagram — بول Urdu TTS system modules

3.9 Activity Diagram

Figure 3.9 shows the complete user activity flow from opening the application through authentication, text input, the standard TTS or voice cloning decision, speech generation, playback, and optional audio download.

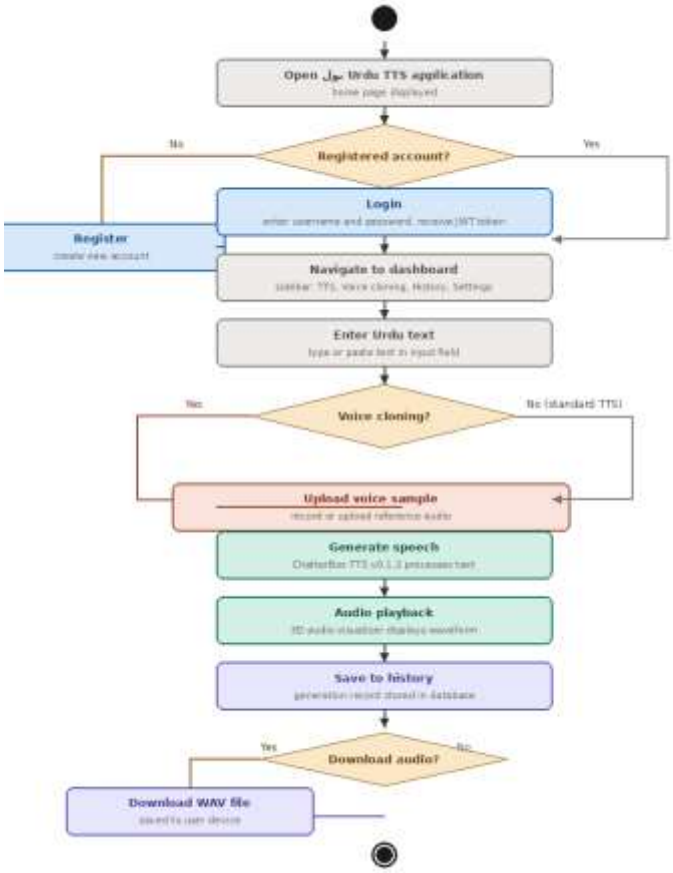

Figure 3.9: Activity diagram — user workflow in بول Urdu TTS

3.10 Dataset Statistics

Table 3.3 summarizes the Kaggle Urdu 20,000-file dataset used for fine-tuning ChatterBox TTS v0.1.2.

Table 3.3: Dataset Statistics

Attribute	Value
Dataset Source	Kaggle — Urdu audio dataset with transcription (20,000 files)
Total Files	20,000 WAV audio files with TSV transcriptions
Duration Filter	1.0 – 10.0 seconds per clip (after filtering)

Files After Filtering	17,121 valid files (as logged during preprocessing on Google Colab)
Sample Rate	24,000 Hz (resampled for ChatterBox TTS compatibility)
Speaker Split	Train: 90% | Validation: 5% | Test: 5% (speaker-level split)
Minimum Samples per Speaker	5 samples (speakers with fewer samples excluded for stability)
Text Normalization	Strong normalization: diacritics removed, Arabic script preserved
Audio Format	WAV (16-bit PCM, resampled to 24 kHz for model compatibility)
Transcription Format	TSV metadata with client_id, path, sentence, and duration fields

3.11 API Endpoint Summary

Table 3.4 summarizes the API endpoints exposed by the backend server.

Table 3.4: API Endpoints

Method	Endpoint	Auth	Description
POST	/register	None	Register a new user account with username, email, and password.
POST	/token	None	Login and receive a JWT access token (OAuth2 Bearer).
GET	/users/me	JWT	Retrieve current authenticated user's profile information.
POST	/tts_simple	JWT	Generate Urdu speech from text input using default ChatterBox TTS voice.
POST	/clone	JWT	Generate Urdu speech with voice cloning using a reference audio sample.
GET	/history	JWT	Retrieve authenticated user's TTS generation history from database.

GET	/health	None	Health check endpoint confirming API and TTS model status.

3.12 Design Decisions

- ChatterBox TTS v0.1.2 Selection: ChatterBox TTS was selected over alternatives such as YourTTS, XTTS-v2, and MMS-TTS due to its state-of-the-art diffusion-based T3 Turbo architecture, built-in voice cloning support via Resemble Perth, and community fine-tuning support. Its 24 kHz output provides high-quality audio suitable for both academic evaluation and commercial applications.
- SQLite for Database: SQLite was chosen for its simplicity, zero-configuration deployment, and adequacy for a single-instance academic deployment. It can be upgraded to PostgreSQL for production scalability.
- JWT Authentication: Stateless JWT authentication was chosen to allow the web frontend to communicate securely with the backend without server-side session management, enabling future horizontal scaling.
- Sidebar Dashboard: A sidebar-based dashboard layout was chosen to give users clear navigation between TTS generation, voice cloning, history, and settings without full page reloads, improving usability.
- Google Colab for Fine-Tuning: Google Colab's free GPU environment (CUDA) was used for fine-tuning to eliminate hardware costs. Training produced fine-tuned weights saved in the safetensors format.

3.13 Web Application Interface

The following screenshots illustrate the actual running web application (بول Urdu TTS). Dashboard screenshots will be inserted here once provided.

Figure 3.10: Home page — hero section with 'Get Started Free' and 'Explore Features' call-to-action buttons. Features section below shows Text-to-Speech, Voice Cloning, and Fast Processing cards.

Figure 3.11: Features page — detailed feature breakdown showing Advanced Urdu TTS (core technology) and Voice Cloning & Customization (personalization) sections with feature bullet points.

Figure 3.12: About page — mission statement, team member cards (Muhammad Bilal: AI & Model Lead, Rohail Ahmad: Full Stack Lead, Fiza Wahab: Data & Design Lead), and supervisors section.

Figure 3.13: Login page — clean card layout with username and password fields and 'Sign In' button. Link to registration for new users.

Figure 3.14: Register page — account creation form with username, email, password, and confirm password fields with 'Sign Up' button.

[INSERT: Dashboard TTS tab screenshot — text input field, Generate button, audio player with 3D visualizer]

[INSERT: Dashboard Voice Cloning tab screenshot — reference audio upload, text input, Clone & Generate button]

[INSERT: Dashboard History tab screenshot — list of past generations with timestamps]